



Oppimistehtävä 2 - Indeksit

Relaatiotietokannat

Jun Fengari, AG4042

Oppimistehtävä 2

Relaatiotietokannat, Jarkko Immonen

5.12.2025

Tietojenkäsittely

Sisältö

1	Lähtötilanteen analyysi.....	2
2	Kyselyiden Tehostaminen	3
3	Kokoteksti-Indeksi	5
4	Reflektio ja tekoälyn käyttö	8
4.1	Reflektio	8
4.2	Tekoälyn Käyttö.....	8
Lähteet		13

1 Lähtötilanteen analyysi

Mitä indeksejä tietokannassa on jo valmiiksi?

Tietokannan indeksien selvittämiseen käytin SHOW INDEXES FROM (taulun nimi) jokaiselle taululle. Sain selville että tietokannassa on Primary Key indeksit, ja Foreign Key (jos taulusta löytyy näitä) indeksit valmiiksi luotuna.

Tyypilliset käyttötävät

Tyypillisiä käyttötapoja kirjastotietokannalle voisi olla:

1. Listaa top 10 lainatut kirjat.
2. Kuinka monta lainausta on tehty Muuramessa 2019 vuoden aikana?
(tai kuinka monta kirjaa Muuramelaiset ovat lainanneet vuoden aikana yhteensä, jos Muuramessa ei ole erillistä kirjastoa).

Toteutus:

1. Listaa top 10 lainatut kirjat.

Tarvittavat liitokset: Nimeke, Nide, LainaustaNide, Lainausta
nimeke.nimeke-nide.nimeke, nide.nide-lainausta.nide.nide, lainausta.nide.lainausta-lainausta.lainausta

```
SELECT nimeke.nimi AS top10kirjat FROM nimeke
INNER JOIN Nide ON nimeke.nimeke = nide.nimeke
INNER JOIN LainaustaNide ON nide.nide = lainausta.nide.nide
INNER JOIN Lainausta ON lainausta.nide.lainausta = lainausta.lainausta
GROUP BY nimeke.nimi
ORDER BY COUNT(lainausta.lainausta) DESC
LIMIT 10;
```

Tässä kyselyssä liitän taulut yhteen nimekkeestä lainaukseen, lasken lainausten määrä order by osassa ja järjestetään group by osassa nimien mukaan, ja sitten valitsen niistä 10 eniten lainatut kirjat (limit 10 & desc).

2. Kuinka monta lainausta on tehty Muuramessa 2019 vuoden aikana?

Tarvittavat liitokset: Asiakas, Lainausta
lainausta.asiakas-asiakas.hlotunnus

```
SELECT asiakas.postitoimipaikka, COUNT(lainausta.lainausta) AS lainaustenlkm FROM Lainausta
INNER JOIN asiakas ON lainausta.asiakas = asiakas.hloTunnus
WHERE lainausta.paiva BETWEEN '2019-01-01' AND '2019-12-31' AND postitoimipaikka = 'Muurame'
GROUP BY asiakas.postitoimipaikka;
```

Tässä kyselyssä liitän asiakas ja lainaustaulun yhteen, ja valitsen where ehdolla 2019 sisältyvät lainauspäivät sekä oikean paikkakunnan.

2 Kyselyiden Tehostaminen

Nykyinen tilanne

1. Ensimmäisen kyselyn EXPLAIN tulokset:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Nide	NULL	index	PRIMARY,nide_nimeke_fk	nide_nimeke_fk	82	NULL	120	100.00	Using index; Using temporary; Using filesort
1	SIMPLE	nimeke	NULL	eq_ref	PRIMARY	PRIMARY	82	kirjasto.Nide.nimeke	1	100.00	NULL
1	SIMPLE	LainausNide	NULL	ref	PRIMARY,lainausnide_nide_fk	lainausnide_nide_fk	4	kirjasto.Nide.nide	6	100.00	Using index
1	SIMPLE	Lainaus	NULL	eq_ref	PRIMARY	PRIMARY	4	kirjasto.LainausNide.lainaus	1	100.00	Using index

Näissä tuloksissa nähdään että:

- Nide-tauluun haetaan koko indeksi foreign keyn avulla, ja arvioitu rivimäärä (120) kertoo, että tämä ei välttämättä ole tehokkain tai nopein tapa hakea tietoa. Extra sarakkeessa näkyy, että "Using index; Using temporary; Using filesort" -merkinnät ovat käytössä, mikä tarkoittaa, että kysely hakee tiedot Nide taulusta indeksin avulla, tallentaa ne väliaikaiseen tauluun ja lajittelee ne sen jälkeen. Kysyn tekoälyltä (todisteet tekoälyn käyttö -osuudessa) onko tämä tehoton tapa suorittaa kysely, mutta sain ymmärryksen, että group by ja aggregaattikyselyissä tämä käsittely on normaalia.
- Nimeke- ja Lainaus-tauluihin käytetään eq_ref-tyyppiä, mikä on paras vaihtoehto liittotilanteissa. Nimeke taulun kohdalla Extra-sarake näyttää null, mikä tarkoittaa että indeksiä voidaan käyttää rivin suoralla haulla ilman lisäoperaatioita.
- LainausNide käyttää tyyppiä ref, mikä tarkoittaa, että haku löytää useita riviä foreign key-indeksiä käyttäen. Sekä LainausNide- että Lainaus-tauluissa nähdään Extra-sarakkeessa että hyödynnetään indeksiä (Using index).

2. Toisen kyselyn EXPLAIN tulokset:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Lainaus	NULL	ALL	lainaus_asiakas_fk	NULL	NULL	NULL	250	11.11	Using where
1	SIMPLE	asiakas	NULL	eq_ref	PRIMARY	PRIMARY	44	kirjasto.Lainaus.asiakas	1	10.00	Using where

- Asiakas-taulu käyttää tyyppinä eq_ref, mikä on nopea ja tehokas, koska se hakee vain yhden rivin primary key-indeksin avulla.
- Sen sijaan Lainaus-taulu käyttää huonointa vaihtoehtoa, ALL, mikä tarkoittaa, että se käy kaikki rivit läpi. Key-sarakkeesta nähdään, että se ei käytä indeksiä lainkaan.

Tässä kyselyssä kummallakin taululla on Extra-sarakkeessa 'Using where', mikä kertoo, että kysely suodattaa tuloksia where-ehdon avulla – mikä on tässä tapauksessa tavallista. Tämä ei välttämättä viittaa tehokuuteen, vaan tehokkuuden kannalta type-sarake kertoo olennaisemman tiedon.

Tehostaminen indeksien avulla & EXPLAIN

Yllä olevien tulosteiden perusteella tiesin, että ainakin toisen kyselyn kohdalla tehostaminen on tarpeellista, sillä siinä näkyy ALL-tyyppi, joka on huonoin vaihtoehto. Ensimmäisen kyselyn kohdalla en ollut varma, mutta epäilin, että Nide-taulun käsittelyä voisi parantaa.

1. Ensimmäisen kyselyn uudet indeksit ja selitykset:

Taulussa on monta liitosta, joten yksi tapa tehostaa voisi olla uuden komposiitti-indeksin lisääminen, joka nopeuttaa liitoksia ja hakua.

Kyselyssä etsitään nimikkeillä niteitä ensimmäisessä joinissa (nimeke.nimeke -> nide.nimeke), joten tätä prosessia voidaan nopeuttaa tekemällä Nide-tilulle komposiitti-indeksin sarakkeelle nimeke, jotta kysely pystyy löytämään vastaavat rivit nopeammin.

```
CREATE INDEX compindx_nide_nimeke ON nide(nimeke);
```

Tässä tuli vastaan ongelma, MySQL ilmoitti virheen, "Duplicate Index", eli tämä liitos on jo optimoitu nide_nimeke_fk-indeksissä, ja siihen ei saanut lisätä toista indeksia.

Sitten kokeilin vielä tehdä komposiitti-indeksin Lainaused-tiluun, joka oli sallittua:

```
CREATE INDEX cmpidx_lainausnide_nide_lainaus ON LainausNide (nide, lainaus);
```

Tässä yritin tehostaa LainausNiden hakua, mutta kun ajoin EXPLAIN uudelleen, huomasin, että tämä vain pahensi tilannetta. Aijempi eq_ref Primary key:n avulla muuttuikin ref-tyyppiä.

Poistin kaikki tekemäni indeksit, ajoin uudelleen EXPLAIN, ja tulos oli aivan erilainen:

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	nimeke	NONE	ALL	PRIMARY	NONE	NONE	NONE	36	100.00	Using temporary; Using filesort
	1	1 SIMPLE	Nide	NONE	ref	PRIMARY,nide_nimeke_fk	nide_nimeke_fk	82	kirjasto.nimeke.nimeke	3	100.00	Using index
	1	SIMPLE	LainausNide	NONE	ref	PRIMARY,lainausnide_nide_fk	lainausnide_nide_fk	4	kirjasto.Nide.nide	6	100.00	Using index
	1	SIMPLE	Lainaus	NONE	eq_ref	PRIMARY	PRIMARY	4	kirjasto.LainausNide.lainaus	1	100.00	Using index

Kysyin tässä vaiheessa tekoälyltä, mitä tapahtui ja miten pystyn palauttamaan tilanteen takaisin alkuperäiseen kuntoon. ChatGPT selitti, että vaikka poistin omat huonot indeksit, MySQL saattoi silti valita uuden strategian taulujen käsittelyyn.

Koska nimeke taulua käytiin kokonaan läpi, päätin tehdä indeksin nimeke taulun nimeke sarakkeeseen sillä ensimmäinen liitos on tehty nimeke sarakkeeseen:

```
CREATE INDEX idx_nimeke_nimeke on nimeke(nimeke)
```

Tämä ei valitettavasti korjannut ALL-tyyppin käyttöä. Kysyin tekoälyltä minkä takia, ja mitä kannattaisi seuraavaksi kokeilla. Sain sen kautta selville että ongelma ei todennäköisesti ole liitoksen kohdassa vain GROUP BY osiossa, jossa ryhmitetään nimi sarakkeen mukaan. Poistin aijemman indeksin ja loin seuraavan nimi sarakkeeseen:

```
CREATE INDEX idx_nimeke_nimi on nimeke(nimi)
```

Tällä sain ALL-tyyppin poistettua Nimeke-tilun käsittelystä, ja seuraavaksi parhaan eli index-tyyppin takaisin.

Tilanteen yleiskatsaus:

Table of EXPLAIN changes

Table	Original	After removing bad indexes	With covering index
Nimeke	eq_ref	ALL	index
Nide	index	ref	ref
LainausNide	ref	ref	ref
Lainaus	eq_ref	eq_ref	eq_ref

Yritin tässä päätellä, oliko alkutilanne parempi kuin lopputilanne. Ymmärsin, että alkutilanne oli

parempi, mutta lopputilanne on kuitenkin parempi kuin se, että olisin jättänyt ALL Nimeke tauluun indeksikokeilujen jälkeen.

2. Toisen kyselyn uudet indeksit ja selitykset:

Ennen aloittamista, tarkistin EXPLAIN-komennolla, onko kyselyn käsittelytapa sama kuin aiemmin, ja huomasin tässä myös, että MySQL valitsi taas erilaisen polun kyselyn suorittamiseen:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	asiakas	HULL	ALL	PRIMARY	HULL	HULL	HULL	37	10.00	Using where
1	SIMPLE	Lainaus	HULL	ref	lainaus_asiakas_fk	lainaus_asiakas_fk	44	kirjasto.asiakas.hloTunnus	7	11.11	Using where

Tässä polussa on edelleenkin ALL-tyyppi käytössä, vain eri kohdassa kun aiemmin, joten aloin tämän perusteella tehostamaan kyselyn käsittelyä.

Kyselyssä suodatetaan Asiakas-tilua WHERE-ehdon avulla, ja koska postitoimipaikalla ei ole indeksia, MySQL joutuu käymään koko taulun läpi, joten päätin kokeilla postitoimipaikan indeksin lisäämistä.

CREATE INDEX idx_asiakas_postipaikka ON asiakas(postitoimipaikka);

Lopputulos:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	asiakas	HULL	ref	PRIMARY,idx_asiakas_postipaikka	idx_asiakas_postipaikka	82	const	7	100.00	Using index
1	SIMPLE	Lainaus	HULL	ref	lainaus_asiakas_fk	lainaus_asiakas_fk	44	kirjasto.asiakas.hloTunnus	7	11.11	Using where

Yhteenveto

Ensimmäisen kyselyn kohdalla yritin ensin tehostaa kyselyssä olevia liitoksia, mutta huomasin, että ne olivat jo indeksoitu automaattisesti foreign key:n avulla. Seuraavaksi selvittelin, miten INDEX-tyyppi toimii ja miksi, ja yritin toisella indeksillä ”korjata” sitä, mutta se ei toiminnut, joten poistin molemmat indeksit. EXPLAIN-tulokset muuttuivat, ja päätin korjata vain ALL-kohdan, joka aiheutti haasteita.

Lopputulos oli, että taulun käsittely ei ollut yhtä tehokas kuin aiemmin.

Opin tästä että MySQL käsittelypäättökset ovat monimutkaisia ja vaihtelevia, ja tässä kyselyssä vaikutusvaltani oli lopulta pieni. Tiedän tulevaisuudessa, että INDEX-tyyppi ei välttämättä ole huono asia, ja sitä ei kannata välttämättä muuttaa, varsinkin jos muiden taulujen käsittely on erittäin tehokasta.

Toinen kysely oli ensimmäisen kyselyn jälkeen erittäin helppo. Korjasin siitä Asiakas-tilun ALL-käsittelytyypin indeksoimalla WHERE-ehdon hakua (eli postitoimipaikka).

Kirjoitin molempien kyselyiden tarkemmat prosessit ja selitykset omiin osiinsa.

3 Kokoteksti-Indeksi

1. Mikä on kokoteksti indeksi?

Kokoteksti indeksi on työkalu, jolla voidaan etsiä tietueita tietokannasta avainsanojen perusteella, toisin kun tavallisesti, jolloin käytetään suoraa merkkijonoverailua. Kokoteksti-indeksissä voidaan käyttää NATURAL LANGUAGE MODE -hakua, joka tulkitsee merkkijonon oikeana fraasina, ja tämä on käytössä oletuksena, jos muuta modifioijaa ei ole määritetty.

WITH QUERY EXPANSION on toinen hakutapa, joka suorittaa haun kahdesti: toisella hakukerralla se etsii lisää dokumentteja, jotka liittyvät ensimmäiseen hakutulokseen.

BOOLEAN MODE on hakutapa, jonka avulla hakua voidaan tarkentaa käyttämällä vertailuoperaattoreita ja hakusymboleita. (Vileikis, 2020)

2. Millainen kokoteksti-indeksi hyödyntäisi kirjaston asiakkaita?

Kokoteksti-indeksi hyödyntäisi asiakkaita mahdollistamalla hakujen tekemisen avainsanojen perusteella, esimerkiksi etsimällä kirjaa tai kirjailijaa tietyillä hakusanoilla. Mielestäni tämä vastaa hyvin asiakkaiden tyypillistä hakutapaa, sillä emme aina muista tarkkoja kirjan tai kirjailijan nimiä. Kirjastomme tapauksessa kokoteksti-indeksi voisi mahdollistaa haun kirjan nimen, kirjailijan nimen, avainsanojen ja/tai kuvauksen perusteella.

3. Kokoteksti-Indeksi - Natural Language Mode

Kokoteksti-indeksi nimeke tauluun:

```
CREATE FULLTEXT INDEX fulltext_nimeke ON nimeke(kuvaus, avainsanat, nimi);
```

Tässä tein FULLTEXT-indeksin nimeke-tauluun, sarakkeisiin kuvaus, avainsanat, ja nimi. Tämä tarkoittaa, että hakua tehdessä MySQL voi etsiä tuloksia kaikista näistä sarakkeista tehokkaasti. Syntaksin lähde (PlanetScale, 2023).

```
SELECT * FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('kevyt dekkari' IN NATURAL LANGUAGE MODE);
```

Tässä hyödynsin äsken luotua indeksiä hakemalla nimeke-taulusta kuvaus-, avainsanat- ja nimi-sarakkeista sanoja 'kevyt dekkari'. IN NATURAL LANGUAGE MODE ei olisi välttämätön, sillä se on oletuksena käytössä, mutta lisäsin sen vaan selkeyden vuoksi.

Sitten lisäsin painokertoimen ja viimeistelin tuloksen:

```
SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('kevyt dekkari'), 2) AS painokerroin
FROM nimeke
WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('kevyt dekkari' IN NATURAL LANGUAGE MODE)
ORDER BY painokerroin DESC
```

Syntaksin lähde (Roman Agabekov, 2025), eli painokerroin saadaan lisäämällä sama MATCH-lause SELECT-osioon, jolloin painokertoimen tulos voidaan nähdä ja järjestää haluamalla tavalla.

Muita kokeiluja:

```
6 • SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('klassinen'), 2) AS painokerroin
7 FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('klassinen' IN NATURAL LANGUAGE MODE)
8 ORDER BY painokerroin DESC
```

Result Grid							
Filter Rows:							
Export: Wrap Cell Content: I A							
	nimeke	nimi	kustantaja	luokitus	kirjailija	kuvaus	avainsanat
▶	2222-3333-444	Esirippu	NOVA	84.2	1	Klassinen salapolisiromaani, jossa jännitys kirist...	dekkari, rikos, Christie
5							
6 •	SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('linnut, puutarha'), 2) AS painokerroin						
7	FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('linnut, puutarha' IN NATURAL LANGUAGE MODE)						
8	ORDER BY painokerroin DESC						

Result Grid							
Filter Rows:							
Export: Wrap Cell Content: I A							
	nimeke	nimi	kustantaja	luokitus	kirjailija	kuvaus	avainsanat
▶	5567-6678-777	Puutarhan linnut	NOVA	55.5	4	Puutarhan linnut – pihan ruokinta, pesintä ja ha...	linnut, puutarha, ruokinta
	0012-1123-222	Euroopan linnut	NOVA	55.5	11	Euroopan linnut – levinneisyys, tuntomerkit ja m...	linnut, Eurooppa, muuttoreitit
	5555-6666-777	Suomen linnut	NOVA	55.5	4	Kattava katsaus Suomen lintulajeihin – tuntome...	linnut, luonto, Suomi
	0000-1111-222	Lintubongarin opas	NOVA	55.5	4	Lintubongarin perusopas – varusteet, havainnoi...	linnut, bongaus, opas
	5556-6667-777	Retkeilijän lintukirja	NOVA	55.5	7	Retkeilijän lintukirja – maastossa tunnistamisen ...	linnut, retkeily, maasto
	5557-6668-777	Suomen metsälinnut	NOVA	55.5	10	Suomen metsälinnut – elintavat, elinalueet ja vu...	linnut, metsä, Suomi

Tässä havaitsin, että sanat, jota etsitään NATURAL LANGUAGE MODE -haulla, pitää olla täsmälleen samoja kuin sanat kuin kirjan nimessä, kuvauksessa, tai avainsanoissa. Tämä haku on hyödyllinen kun etsitään täsmä-tulosta.

Jos etsitään perhoskirjoja, ei voida hakea sanan alulla, kuten ”perho”, koska se ei tuota yhtään tuloksia. Tällaisissa tapauksissa voidaan hyödyntää BOOLEAN MODE-hakua.

4. Boolean Mode

Voidaan käyttää * wildcard operaattoria hakemaan sanan alkua:

```
6 • SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('perho*' IN BOOLEAN MODE), 2) AS painokerroin
7 FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('perho*' IN BOOLEAN MODE)
8 ORDER BY painokerroin DESC
```

Result Grid Filter Rows: Export: Wrap Cell Content:							
	nimeke	nimi	kustantaja	luokitus	kirjailija	kuvaus	painokerroin
▶	0001-1112-222	Perhoset ja muut hyönteiset	NULL	55.5	6	Perhosten ja muiden hyönteisten esittely – elink...	3.15

```
SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('perho*' IN BOOLEAN MODE), 2) AS painokerroin
FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('perho*' IN BOOLEAN MODE)
ORDER BY painokerroin DESC;
```

Tämä on hyödyllistä, jos muistetaan vain osan kirjan nimestä tai kuvauksesta, tai halutaan hakea kaikkia kirjoja, jotka voivat kuulua tiettyyn aihepiiriin. Hakija ei esimerkiksi tiedä, onko avainsana täsmälleen ”perhonen” vai ”perhoset”.

Tässä voidaan etsiä kaikki tietokantoja koskevat kirjat ilman Python-aiheisia tuloksia:

```
6 • SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('+tietokanta -python' IN BOOLEAN MODE), 2) AS painokerroin
7 FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('+tietokanta -python' IN BOOLEAN MODE)
8 ORDER BY painokerroin DESC
```

Result Grid Filter Rows: Export: Wrap Cell Content:							
	nimeke	nimi	kustantaja	luokitus	kirjailija	kuvaus	painokerroin
•	7778-8889-999	Tietokanta ABC	NULL	11.1	2	Tietokanta ABC – käsitteet, normalisointi ja SQL...	2.21
•	1111-2222-333	SQL-Opas	NULL	11.1	2	Tietokantojen käsittelyn perusteet SQL-kielillä. ...	0.74
•	7777-8888-999	Tietokannat käytännössä	NULL	11.1	2	Tietokantojen käytön käytännönläheinen opas ...	0.74

```
SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('+tietokanta -python' IN BOOLEAN MODE), 2) AS painokerroin
FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('+tietokanta -python' IN BOOLEAN MODE)
ORDER BY painokerroin DESC;
```

Tämä on hyödyllistä tulosten suodattamiseen. Tässä tilanteessa hakija haluaa lukea tietokannoista mutta ei Pythonista.

Tässä yritän asettaa vähemmän arvoa sanalle ”historia” ja enemmän sanalle ”media”, mutta se ei toiminut kuten odotin:

```
6 • SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('>media <historia' IN BOOLEAN MODE), 2) AS painokerroin
7 FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('>media <historia' IN BOOLEAN MODE)
8 ORDER BY painokerroin DESC
```

Result Grid Filter Rows: Export: Wrap Cell Content:							
	nimeke	nimi	kustantaja	luokitus	kirjailija	kuvaus	painokerroin
▶	6666-7777-888	TV:n historia	NULL	66.6	5	Television kehityksen päälinjat ja keskeiset murr...	3.49
•	1113-2224-333	TV-tekniikan historia	NULL	66.6	9	TV-tekniikan historia – lähetysjärjestelmät, näyt...	2.49
•	6667-7778-888	TV ja yhteiskunta	NULL	66.6	8	Television yhteiskunta: vaikutteet, valta ja me...	2.16
•	6678-7789-888	TV-kulttuuri	NULL	66.6	5	TV-kulttuuri – katsoisuus, fanitus, politiikka ja po...	2.16
•	1112-2223-333	Televisionviihteen vuosisata	NULL	66.6	5	Televisionviihteen kehitys ja ilmiöt vuosikymment...	0.16

```
SELECT *, ROUND(MATCH(kuvaus, avainsanat, nimi) AGAINST('>media <historia' IN BOOLEAN MODE), 2) AS painokerroin
```


FROM nimeke WHERE MATCH(kuvaus, avainsanat, nimi) AGAINST('>media <historia' IN BOOLEAN MODE)

ORDER BY painokerroin DESC

Tulokset toimii niin, että:

- kaikista relevanteimmat tulokset sisälsivät sekä "media" että "historia"
- seuraavaksi tulokset, joissa oli vain "media"
- heikoin relevanssi oli tuloksissa jotka sisälsivät vain "historia" sana.

Tämä saattaa olla käytännöllistä suurissa kokoelmissa, joissa hakija etsii tarkempia tuloksia avainsanojen avulla.

4 Reflektio ja tekoälyn käyttö

4.1 Reflektio

Tämän oppimistehtävän avulla opin paljon uutta indeksoinnista.

Opin hahmoittamaan MySQLin tapaa käydä tietokantojen tauluja läpi, ja että sen käsittelypäädotukset ovat yllättävän monimutkaisia. Indeksien tehostamisen kohdalla huomasin, että MySQL saattaa muuttaa käsittelytapoja vaikka tauluun tai tietokantaan ei tehty pysyviä muutoksia.

Tehtävänteon aikana huomasin myös, että jo valmiiksi luodut indeksit primary- ja foreign key:n avulla auttavat näissä mittakaavoissa, mutta isommilla datamäärillä tehokkuuden ja indeksien tarve kasvaa huomattavasti. Tehtäviä olisi myös ollut mahdollisesti helpompi hahmottaa, jos olisi nähnyt käytännössä paremmin, miten isot datamäärät käyttäytyvät tehokkuuden kannalta.

Oppimistehtävän alussa oli vaikeaa hahmoittaa, mihin indeksejä asennetaan ja miksi, mutta nyt oppimistehtävän lopussa se on selkeytynyt. Ensimmäisessä osiossa kyselyiden luominen oli jälleen haasteellista, varsinkin sellaisten, jotka sopisivat indeksoitavaksi.

Toisessa osiossa, varsinkin ensimmäisen kyselyn kohdalla, tuli paljon haasteita. Siinä oli vaikeaa ymmärtää, mitä siitä olisi voinut tehostaa, sillä indeksityyppi on yleisin käyttötapa tämän tyylliselle kyselylle. Tämän kyselyn ja indeksien läpikäymisen kautta selvisi enemmän tietoa SQL:n käsittelytavoista, ja normaalien indeksien teko helpottui heti jälkikäteen.

FullText-indeksi oli hyvin selkeä, ja liitetyt artikkelit auttoivat tosi paljon hahmottamisessa, ja muissa lähteissä oli myös paljon hyödyllistä tietoa ja apua syntakseihin. Selkeyttä auttoi, että FullText-indeksin käyttöä pystyi liittämään hyvin oikean elämän esimerkkeihin, eli esimerkiksi siihen, miten hakupalkkeja tyypillisesti käytetään hakemalla ja suodattamalla tuloksia.

Käytin tämän oppimistehtävän tekemiseen vajaat 2 viikkoa, yhteensä noin 13 tuntia.

4.2 Tekoälyn Käyttö

Tässä oppimistehtävässä käytin tekoälyä moneen tarkoitukseen. Ensin käytin sitä tehtävänannon selittämiseen ja vaikeiden konseptien avaamiseen. Tehtävissä käytin sitä ongelmien ymmärtämiseen, ratkaisuvaihtoehtojen löytämiseen, MySQLin käsittelytehokkuuden ja -tyylin ymmärtämiseen, sekä loogisten ongelmien ja liitoksien merkityksien selvittämiseen indeksien kontekstissa.

Toisessa osiossa ensimmäisen kyselyn kohdalla käytin tekoälyä eniten, sillä kohtasin siinä monimutkaisia ongelmia. Viimeisessä osiossa en käyttänyt tekoälyä ollenkaan.

Lopussa käytin tekoälyä kieliasun tarkistamiseen.

Alla on kuvia keskusteluista, jotka liittyvät tehtävien eri vaiheisiin.

Ensimmäisessä osassa pohdin että voinko käyttää päivämääriä, sillä dokumentaatioissa mainittiin että indeksit ei toimi kun käytetään funktioita. Chatgpt selitti että saa käyttää päivämääriä vertailussa mutta ei seuraavia funktioita kyselyssäni:

Summary — You CAN use dates if you follow this rule

✓ Allowed (index works)

- `WHERE dateColumn >= '2024-01-01'`
- `WHERE dateColumn BETWEEN ...`
- `WHERE dateColumn < CURDATE()`
- `WHERE dateColumn > NOW()`

✗ Avoid (index won't be used)

- `WHERE YEAR(dateColumn) = 2024`
- `WHERE MONTH(dateColumn) = 5`
- `WHERE DATE(dateColumn) = CURDATE()`

Ensimmäisen osan toisessa kyselyssä kysyin ensin onko kyselyni oikein ja sopiva indeksin tekoon, ChatGPT ehdotti GROUP BY lisäystä niin kysyin minkä takia:

✦ So why *should* it need GROUP BY?

Logically, the moment you mix:

- aggregated values → `COUNT(*)`
- non-aggregated values → `asiakas.postitoimipaikka`

SQL standard requires grouping because:

✦ SQL needs to know how to collapse multiple rows into one.

Even though in this specific case all rows have the same value `'Muurame'`, SQL doesn't automatically know that unless the DB engine allows nonstandard behavior.

🧠 Short answer:

- It works *without* `GROUP BY` because your SQL engine allows nonstandard queries (likely MySQL/MariaDB).
- It *should logically* need `GROUP BY`, because you're selecting both aggregated and non-aggregated columns.
- If you move to PostgreSQL or SQL Server, this query will break.

Toisessa osassa kysyin mitä “Extra” tyyppi tarkoittaa Nide taulun analyysin kohdalla, ja mitä tarkoittaa infossa mainitut extra, filesort, ja temporary:

★ What it means together

✓ Step 1: MySQL reads rows from Nide using an index

(good — efficient lookup)

✓ Step 2: MySQL creates a temporary table

(to compute aggregates or groups)

✓ Step 3: MySQL sorts that temporary table manually

(ORDER BY that cannot use an index)

🧠 Easy analogy

It's like this:

"I looked up all needed book copies using the index — fast!

Then I wrote the results on a temporary sheet of paper.

Then I sorted that sheet by hand."



✦ Is this bad?

Not necessarily.

Using index → great

Using temporary → common for grouped reports

Using filesort → normal if you're sorting aggregated data

Your query:

```
sql
SELECT nimeke.nimi, COUNT(*) AS lainauslkm
...
GROUP BY nimeke.nimi
ORDER BY lainauslkm DESC
```

Copy code

always produces temporary + filesort, even with perfect indexing.

So:

- ✦ It does **NOT** mean your query is inefficient
- ✦ It means your query is doing the kind of work that requires grouping and sorting
- ✦ This is *expected* for reporting-type queries



Seuraavaksi selvitin pitkään, että voiko index-tyyppiä parantaa tässä tilanteessa:

This happens when:

- MySQL is grouping or ordering large data sets.
- The index being used does not help filter rows — it just helps avoid a full table scan.
- The relevant columns are missing from the index.

Because your first query:

```
sql
GROUP BY nimeke.nimi
ORDER BY lainauslkm DESC
```

MySQL may need to:

- Read all rows,
- Group them by title,
- Count total loans,
- Sort by that count.

This often causes:

```
pgsql
Using index; Using temporary; Using filesort
```

Which is normal for aggregate queries.

◆ Can you improve `index` to `ref` or `eq_ref`?

Possibly — but only if one of these becomes true:

- ✓ The join condition can uniquely identify rows
- ✓ There is an index on the join column
- ✓ MySQL can use that index to *limit* rows, not just process them

★ When could it improve?

If you add a composite index such as:

```
makefile
(idx: nide.nimeke, lainaus.lainaus)
```

and your grouping + ordering lined up *exactly* with the index, MySQL could skip temporary tables.

But in this query:

- You are sorting by an aggregated count (`ORDER BY lainauslkm DESC`)
- That value does **not exist in the raw tables**
- So MySQL must compute it in a temporary table first

Therefore — optimization is limited.

Ennen kuin aloin kirjoittamaan indeksejä, kysyin tekoälyltä muistusta syntaksista ja lisäselitystä kompossiittiindekseistä (jonka tekoäly mainitsi) ja loogisuudesta, ja sitten myöhemmin myös että minkä

takia tietty indeksi toimii tietyllä tavalla:

◆ **First: What does a CREATE INDEX statement mean?**

The general pattern is:

sql

Copy code

```
CREATE INDEX index_name
ON table_name (column1, column2, ...);
```

This means:

- Create a new index (a fast lookup structure, like a phone book)
- Store it for the specified `table`
- Order or organize the index based on the listed columns

Indexes **do not change data** — they just speed up searching and joining.

What does it mean?

- We are creating an index on the `nide` table.
- The index is ordered by the columns:
 1. `nimeke`
 2. `nide`

Why those two?

Because in your big query:

sql

Copy code

```
INNER JOIN Nide ON nimeke.nimeke = nide.nimeke
```

MySQL often needs to:

- Find all Nide-rows for a specific `nimeke`
- And then proceed row by row

So with this index:

- MySQL can jump **directly** to all rows where `nide.nimeke = x`
- No full table scan needed
- The second column (`nide`) helps MySQL navigate or group results even faster



It's not joining columns together

◆ **Another way to picture it**

Imagine two bookshelves:

- Shelf A = Nimeke
- Shelf B = Nide

A JOIN is like matching items from both shelves.

An INDEX is like adding a label system to shelf B so you can:

- Quickly find matching items on shelf B
- When you already know what you're searching for from shelf A

But the index is still **physically only on shelf B**.

◆ **Short summary**

Concept	What it does	Applies to	
JOIN	Combines data across tables	Multiple tables	
INDEX	Speeds up searching in one table	One table only	
Composite Index	Multi-column index within one table	Still one table	

Toisen osion, ensimmäisen kyselyn viimeisessä kohdassa kysyin minkälaista indeksiä voisin harkitää poistattaeksi ALL-tyyppin käytön:

- Look at the **JOIN condition** involving *nimeke*. What column from *nimeke* is being used to match rows from other tables?
- The **type = ALL** usually means the optimizer can't use an index to find matching rows. Ask yourself: *Which column is it searching through on nimeke?*
- Check whether that column is already indexed. If it's a **primary key** or **unique key**, it's already indexed. If it's something else... maybe it isn't.
- Think about which table in your query starts the execution plan. The planner often begins with the table that has the best selective index for the first join.

Monen eri indeksikokeilun yhteydessä yritin ymmärtää miksi indeksini eivät poista ALL-tyyppin käyttöä:

The true reason:

You **GROUP BY** a column that is **NOT** indexed (nimi)

and MySQL must read the entire table to evaluate the groups.

This has **nothing** to do with the join column.

So:

Indexing the join column is correct,

but it cannot fix the **GROUP BY** scan.

To remove the ALL type, you must also index:

```
sql
nimi
```

Because your query groups by `nimi`.

.I.

Lähteet

Käytin useammassa kohdassa kurssin Indeksi materiaalia lähteenä (Jarkko Immonen, ei pvm.).

Jarkko Immonen. (ei pvm.). *4_indeksointi*. Noudettu 17. marraskuuta 2025, osoitteesta

https://tiko.jamk.fi/~imjar/reltk/4_indeksointi.html

PlanetScale. (2023, maaliskuuta 9). *Fulltext indexes—MySQL for Developers—PlanetScale*. Planetscale.

<https://planetscale.com/learn/courses/mysql-for-developers/indexes/fulltext-indexes>

Roman Agabekov. (2025, toukokuuta 15). *Comprehensive Guide to MySQL Full-Text Indexing: Examples,*

Creation, and Performance Optimization. Releem. <https://releem.com/blog/comprehensive-guide-mysql-full-text-indexing>

Vileikis, L. (2020, marraskuuta 13). Full-Text Searches in MySQL: The Good, the Bad and the Ugly. *Sever-*

alnines. <https://severalnines.com/blog/full-text-searches-mysql-good-bad-and-ugly/>

